

SEQUENCE TYPE

A sequence is an ordered collection of similar or different data items. Using sequence, Multiple values can be stored in the data type in an efficient manner. There are different types of sequence data type such as

- i) Strings
- ii) List

String

A string is a sequence of one or more characters (Unicode characters) inside single, double or triple quotes. Indexing is used to reference individual characters of a String and also allows negative address references to access characters from the back of the String, e.g. -1 refers to the last character, -2 refers to the second last character and so on. Only Integers are allowed to be passed as an index, float or other types will cause a `TypeError`.

The following is an example of String assignment:

```
My_name= "John Kimani"
```

Updation or deletion of characters from a String is not allowed. This will cause an error because item assignment or item deletion from a String is not supported i.e. Strings are immutable and elements of a String cannot be changed once it has been assigned. Only new strings can be reassigned to the same name. In python there is no character data type, a character is just a string of length one and is represented by `str` class.

Example

```
# Python Program for String Manipulation
# Creating a String with single Quotes, double quotes, tripple quotes
name1 = 'John'
name2 = "Kimani"
# Triple Quotes allows multiple lines
name3 = '''Welcome
        To
        USIU-A'''
print (name1)
print (name2)
print (name3)
```

Indexing can be used to print individual elements as shown below:

```
#printing first character
print("Printing First Character: ",name1[0])
#printing last character
print("Printing Last Character: ", name1[-1])
```

Note that strings are immutable as shown below

```
#updating a single character
name2[2] = 'N'
#Cannot Update because strings are immutable
```

In case you have need to use single quote (‘), double quote (“) or back slash you need to use escape sequences as shown below:

```
# Printing Paths with the use of Escape Sequences
path = "C:\\Python\\programs\\"
print("\nEscaping Backslashes: ")
print(path)
```

String Manipulation Functions

To manipulate strings in Python, The following built-in methods can be used

Method	Description
capitalize()	Converts the first character to upper case
count()	Returns the number of times a specified value occurs in a string
swapcase()	changes the lowercase characters into uppercase and vice versa. After execution, it returns the modified string
find()	Searches the string for a specified value and returns the position of where it was found. Returns -1 if substring isn't found
format()	Formats specified values in a string
index()	Searches the string for a specified value and returns the position of where it was found. Returns an exception if substring isn't found
islower()	Returns True if all characters in the string are lower case
isnumeric()	Returns True if all characters in the string are numeric
endswith()	Returns true if the string ends with the specified value
isalpha()	Returns True if all characters in the string are in the alphabet
islower()	Returns True if all characters in the string are lower case
isupper()	Returns True if all characters in the string are upper case
join()	Joins the elements of an iterable to the end of the string
lower()	Converts a string into lower case
replace()	Returns a string where a specified value is replaced with a specified value
split()	Splits the string at the specified separator, and returns a list
upper()	Converts a string into upper case
startswith()	Returns true if the string starts with the specified value
strip()	Returns a trimmed version of the string
len()	The function takes a string as input argument and returns the length of the string

Examples

A program to split a string into various parts at the space character:

```
myString = "Hello, string methods ! "  
myString.split()
```

A program to find the frequency of a character in a string

```
word = "Hello World"  
print("The string is:",word)  
character="l"  
print("The character is:",character)  
frequency=word.count(character)  
print("The frequency of the character in the string is:",frequency)
```

Using join function to join multiple strings into one string using a specified separator

```
" " . join( ["Number" , "one" , " " , "the" , "Campus"] )
```

List

The List is an ordered sequence collection of data items (elements). It is one of the most flexible and frequently used data type in Python. All the items in a list are not necessary of the same data type i.e. lists are similar to arrays but they are not homogeneous which makes it the most powerful tool in Python. A single list may contain Data Types like Integers, Strings, as well as Objects.

Lists are mutable and they cannot be modified once created. They are also ordered and have definite count. The list index starts with 0 and duplication of elements is possible in list. The lists are implemented by list class. Lists in Python can be created by just placing the sequence inside the square brackets []. Unlike Sets, list doesn't need a built-in function for creation of list.

Creating Lists

The list class defines lists. To create a list, you can use list's constructor, as follows:

```
list1 = list() # Create an empty list
list2 = list([2, 3, 4]) # Create a list with elements 2, 3, 4
list3 = list(["red", "green", "blue"]) # Create a list with strings
list4 = list(range(3, 6)) # Create a list with elements 3, 4, 5
```

You can also create a list by using the following syntax, which is a little simpler:

```
list1 = [] # Same as list()
list2 = [2, 3, 4] # Same as list([2, 3, 4])
list3 = ["red", "green"] # Same as list(["red", "green"])

# Creating a Multi-Dimensional List (By Nesting a list inside a List)
List = [['Welcome', 'To'], ['USIU-A']]
print("\n Multi-Dimensional List: ")
print(List)
```

Index Operator []

An element in a list can be accessed through the index operator, using the syntax: `myList[index]`

List indexes are 0 based i.e. they range from 0 to `len(myList)-1`

```
# accessing a element from the list using index number
numbers= [10,20,30]
print("Accessing element from the list")
print(numbers[0])
print(numbers[2])
```

```
# accessing a element using negative indexing
print("Accessing element using negative indexing")
# print the last element of list
print(numbers[-1])
# print the third last element of list
print(numbers[-3])
```

`myList[index]` can be used just like a variable, it is also known as an indexed variable. E.g. the following code adds the values in `myList[0]` and `myList[1]` to `myList[2]`

```
Numbers[2]=numbers[0]+numbers[1]
```

The concatenation (+) and repetition (*) Operators

You can use the concatenation operator (+) to join two lists and the repetition operator (*) to replicate elements in a list. The following are examples of using the operators:

```
list1 = [2, 3]
list2 = [1, 9]
list3 = list1 + list2
print (list3)
```

```
list4 = 3 * list1
print (list4)
```

List Built-in Methods

The following are some of methods used in list

Method	Description
append(x)	To add item x to the end of the list “list1”
reverse()	Reverse the order of the element in the list “list1”
sort()	To sort elements in the list
reverse()	To reverse the order of the elements in list1.
pop(i)	Removes the element at the given position and returns it. The parameter i is optional. If it is not specified, list.pop() removes and returns the last element in the list.
remove(x)	Removes the first occurrence of element x from the list.

Addition of Elements in the List

```
List1=[]
list1.append(1)
list1.append(2)
list1.append(4)
print("List after Addition of Three elements: ")
print(list1)
```

Addition of Element at specific Position (using Insert Method)

```
list1.insert(3, 12)
list1.insert(0, 'USIU-A')
print("List after performing Insert Operation: ")
print(list1)
```

```
list3 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
```

```
print("Initial List: ")
```

```
print(list3)
```

Removing elements from List using Remove() method

```
list3.remove(3)
```

```
list3.remove(4)
```

```
print("List after Removal of two elements: ")
```

```
print(list3)
```

```
list3.pop()
```

```
print("List after popping an element: ")
```

```
print(list3)
```

Removing element at a specific location from the using the pop() method

```
list3.pop(2)
```

```
print("List after popping a specific element: ")
```

```
print(list3)
```

Traversing Elements in a for Loop

The elements in a Python list are iterable. Python supports a convenient **for** loop, which enables you to traverse the list sequentially without using an index variable. For example, the following code displays all the elements in the list **myList**:

```
myList=range(10)
for u in myList:
    print(u, end=" ")
```

You can still have use an index variable if you wish to traverse the list in a different order or change the elements in the list. E.g. following code displays the elements at odd numbered positions.

```
for i in range(0, len(myList), 2):
    print(myList[i], end=" ")
```

Example

Write a python program that inputs five numbers into a list and the computes the sum and average. You program should display numbers above average

```
NUMBER_OF_ELEMENTS = 5 # For simplicity, use 5
numbers=[] # Create an empty list
sum = 0
for i in range(NUMBER_OF_ELEMENTS):
    value = eval(input("Enter a new number: "))
    numbers.append(value)
    sum += value

average = sum / NUMBER_OF_ELEMENTS

count = 0 # The number of elements above average
for i in range(NUMBER_OF_ELEMENTS):
    if numbers[i] > average:
        count += 1

print("Average is", average)
print("Number of elements above the average is", count)
```

List Slicing [start : end]

The index operator allows you to select an element at the specified index. The slicing operator returns a slice of the list using the syntax list[start : end].

The slice is a sublist from index start to index end – 1. The following are some examples:

```
list1 = [2, 3, 5, 7, 9, 1]
Print (list1[2 : 4])
```

The starting index or ending index may be omitted. In this case, the starting index is 0 and the ending index is the last index. E.g.

```
print (list1[ : 2])
print (list1[3 : ])
```

You can also use a negative index in slicing as shown in the following example

```
print (list1[1 : -3])
print (list1[-4 : -2])
```

List Comprehensions

List comprehensions provide a concise way to create a sequential list of elements. A list comprehension consists of brackets containing an expression followed by a **for** clause, then zero or more **for** or **if** clauses. The list comprehension produces a list with the results from evaluating the expression. The following are examples:

```
list1 = [x for x in range(5)]
print (list1)
list2 = [0.5 * x for x in list1]
print (list2)
list3 = [x for x in list2 if x >= 1.5]
print (list3)
```

Inputting Lists

You may often need code that reads data from the console into a list. You can enter one data item per line and append it to a list in a loop. For example, the following code reads ten numbers *one per line* into a list.

```
lst = [] # Create a list
print("Enter 5 numbers: ")
for i in range(5):
    lst.append(eval(input()))
```

Sometimes it is more convenient to enter the data in one line separated by spaces. You can use the string's `split` method to extract data from a line of input. E.g. the following code reads ten numbers separated by spaces from one line into a list.

```
s = input("Enter 5 numbers separated by spaces from one line: ")
items = s.split() # Extract items from the string
lst = [eval(x) for x in items] # Convert items to numbers
for number in lst:
    print (number)
```

Invoking `input()` reads a string. Using `s.split()` extracts the items delimited by spaces from string `s` and returns items in a list. The last line creates a list of numbers by converting the items into numbers.